Inside the xv6 File System

Lab 6 · Track B: build your own small OS (xv6) · 90 minutes

Goal

The xv6 file system has the same ideas as ext4 (inodes, directories as tables, hard links, a journal) in a few hundred readable lines. Today you read what xv6 shows you, write your own stat program, measure the limits written in kernel/fs.h, and finally write find, which walks the directory tree recursively.

Before you start

```
bash os2026-setup.sh --check     # toolchain, QEMU and ~/xv6-riscv are ready
```

- Use your ~/xv6-riscv tree. Every new program: user/NAME.c plus a line $U/_NAME\ in UPROGS (TAB at the start, nothing after the backslash). Exit QEMU with Ctrl-A then X.
- Tip: the file system image is rebuilt by make. To start from a clean disk: make clean ; make qemu

Part 1. Read what xv6 shows you (~15 min)

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
...
console        3 25 0
$ mkdir a
$ ls a
```

Each line is: name, type, inode number, size in bytes. Open kernel/stat.h to decode the type.

Observe (write in notes.md)

(1) What do types 1, 2 and 3 mean (kernel/stat.h)? (2) Which inode number does the root directory have, and why are . and .. the same there? (3) An empty directory a has size 32. A directory entry has a fixed size. How big is one entry, and which two entries does a contain?

Part 2. Write mystat (~25 min)

The stat() library call fills a struct stat (kernel/stat.h: dev, ino, type, nlink, size). Complete user/mystat.c:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  // TODO: return "directory", "file" or "device" for T_DIR, T_FILE, T_DEVICE
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    // TODO: call stat(argv[i], &st); on error print to fd 2 and continue
    // TODO: print  NAME: TYPE, inode N, links N, size N bytes
  }
  exit(0);
}
```

Then use it to watch hard links at work:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 25, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
$ rm README
$ mystat r2
$ cat r2 | wc
```

Observe (write in notes.md)

(1) Paste the outputs after ln and after rm. What changed and what did not? (2) Find the line in kernel/sysfile.c (sys_unlink) that decrements the link count. When are the data blocks freed? (3) xv6 has no symbolic links and no permissions. Look at struct dinode in kernel/fs.h: which fields does an xv6 inode have?

Part 3. Measure the limits (~15 min)

```
grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ"
kernel/fs.h
```

```
# while building, make prints the disk layout (look for the nmeta line)
make clean ; make fs.img 2>&1 | grep nmeta
```

```
$ mkdir averyveryverylongname
$ ls
```

Observe (write in notes.md)

(1) What does ls show for averyveryverylongname, and which constant explains it? (2) Compute the largest file xv6 can hold: 12 direct blocks plus one indirect block that holds block numbers of 4 bytes each, blocks of BSIZE bytes. Show the calculation. (3) From the nmeta line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?

Part 4. Write find (~30 min)

user/find.c: find <dir> <name> prints every path under <dir> whose last component is <name>. This is the find exercise of the MIT 6.1810 util lab.

- Start from user/ls.c: it already shows how to open a directory, read struct dirent entries one by one, build the full path in a buffer, and call stat on it.
- Skip entries with inum == 0 (free slots), and never recurse into "." and "..".
- Compare names with strcmp. Close every directory you open (only 16 open files per process).

Test on a fresh disk (make clean ; make qemu). The order of the lines may differ:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

Stretch (optional)

Add a third argument: find . r2 -n prints, next to each path, its inode number. Which of the three r2 files are the same file as cat?

Wrap-up and deliverable (~5 min)

You submit:

- user/mystat.c, user/find.c, the Makefile with your UPROGS lines, and your lab report (see below), which includes the answers from notes.md.

Lab report (required):

- Every lab must be accompanied by a written report: what you did, the answers to all "Observe" boxes with the commands and outputs (or screenshots) that support them, and a short conclusion.
- Labs 5 and 6 may be combined into a single report. If you do this, give each lab its own clearly labelled section and submit the report once, together with the Lab 6 code.
- Everything you submit (the report and any code) must also be pushed to your Git repository.